StreamsML User and Developer Manual

April 22, 2026

Contents

1 User Instructions **3**
1.1 Software Build Instructions . . . 3
1.1.1 Setup . . .
1.1.2 Build . . .
1.2 Configuration and Running a Standard Simulation . . . 5
1.2.1 Configuring and Running a Standard Simulation . . .
1.3 Evaluation Run (Relevant for LearningBased Simulations) . . . 7
1.4 Simulation Output . . . 8
1.4.1 OpenLoop Output . . .
1.4.2 Classical Control Output . . .
1.4.3 LearningBased Output . . .
1.4.4 Preserving your data . . .
1.5 Analyzing a Simulation . . . 9
1.6 Visualizing Simulations and Results . . . 10
1.7 Modeling . . . 11

2 Developer Instructions **12**
2.1 Adding Control Methods . . . 12
2.1.1 Justfile and Rust Modification . . .
2.1.2 OpenLoop Control Methods . . .
2.1.3 Classical Control Methods . . .
2.1.4 LearningBased Methods . . .
2.1.5 Wrapping additional STREAmS Functions . . .
2.2 Adding Analysis Methods . . . 18

3 Developer Information **19**

A.1 File Organization **20**
A.1.1 h5 file schema . . . 20

B.2 Control Methods **21**
B.2.1 Deep Q Learning (DQN) . . . 21
B.2.2 Deep Deterministic Policy Gradient (DDPG) . . . 21
B.2.3 Proximal Policy Optimization (PPO) . . . 21

C.3 Analysis Methods **21**
C.3.1 Proper Orthogonal Decomposition (POD) . . . 21
C.3.2 Dynamic Mode Decomposition (DMD) . . . 21

Manual Structure

User Instructions: Software use without modification

Developer Instructions: Software use with user-provided control and analysis methods

User Information: A comprehensive overview of how the software functions, for those who would like to fundamentally modify the software.

This document is split into three sets of instructions. Section 1, User Instructions, provides instructions for individuals who wish to use the existing software, Section 2, Developer Instructions, provides the instructions for individuals who wish to test their own algorithms and methods using the software, and finally Section 3, Developer Information, provides a comprehensive explanation of the functioning of the software and will be a useful reference and highly suggested reading for those who wish to make fundamental changes beyond testing new control and analysis methods.

Summaries are boxed in purple

Actionable and essential-to-know items are boxed in yellow

```
Justfile entries, Rust, and Python code are boxed in grey
```

1 User Instructions

All user actions take place in the streams-utils subdirectory of StreamsML

1.1 Software Build Instructions

- Setup
  - Check your OS' minimum CUDA driver requirements
  - Execute install_dependencies.sh to add required paths to your bash file, create a virtual environment, and install local software dependencies therein.
- Build
  - Sequentially run "just nv", "just base", "just build" to create the final streams.sif container in which the software will run.

After system installs into a virtual environment and the addition of two paths to one's bashrc, all handled via the execution of install_depedencies.sh, StreamsML is built with three commands, "just nv", which creates an nvidia docker image, "just base", which bootstraps nv.sandbox and installs an Ubuntu OS and necessary software into the container, and finally "just build" which compiles the Fortran and Cargo code.

1.1.1 Setup

Components of modSTREAmS build:

- NVIDIA HPC SDK: A docker image, provided by Nvidia, to contain the libraries and tools required for GPU-accelerated, portable HPC modeling and simulation applications.
- base.apptainer: An apptainer image used to install required software, such as PyTorch, h5py, and an Ubuntu OS.
- build.apptainer: An apptainer image used to compile the Fortran and Rust code, and bind the Python code to the built streams.sif container.

System Requirements In order to use Nvidia HPC SDK, version 24.7-devel-cuda_multi-ubuntu22.04, the following requirements must be met:

Table 2. HPC SDK Platform Requirements

Architecture	Linux Distributions	Minimum gcc/glibc Toolchain	Minimum CUDA Driver
x86_64	RHEL/CentOS 7.3 - 7.9 RHEL/CentOS/Rocky 8.0 - 8.10 RHEL/Rocky 9.2 - 9.4 OpenSUSE Leap 15.4 - 15.4 SLES 15SP2, 15SP3, 15SP4 Ubuntu 18.04, 20.04, 22.04, 24.04 Debian 11	C99: 4.8 C11: 4.9 C++03: 4.8 C++11: 4.9 C++14: 5.1 C++17: 7.1 C++20: 10.1	450.36.06
aarch64	RHEL/CentOS/Rocky 8.0 - 8.10 Rocky 9.2 - 9.3 Ubuntu 20.04, 22.04, 24.04 SLES 15SP4 Amazon Linux 2023	C99: 4.9 C11: 4.9 C++03: 4.9 C++11: 4.9 C++14: 5.1 C++17: 7.1 C++20: 10.1	450.36.06

Complete tag information can be found at https://docs.nvidia.com/hpc-sdk/archive/24.7/hpc-sdk-release-notes/index.html
Other tags can be found here: https://catalog.ngc.nvidia.com/orgs/nvidia/containers/nvhpc/tags

After checking the system requirements above, clone the StreamsML repository

```
git clone https://github.com/fluid-Dynamics-Group/StreamsML.git --depth 1
```

then execute install_dependencies.sh to define environment variables, create a virtual environment, and install dependencies.

```
chmod +x install_dependencies.sh
bash install_dependencies.sh
```

1.1.2 Build

Now that the setup has been completed, the remaining build becomes a simple sequence of three terminal commands.

From the streams-utils directory, run the following commands:

```
just nv
just base
just build
```

after which the software is built and all further interaction with the software is completed by editing the flags in the Justfile, a document found in that same streams-utils directory.

1.2 Configuration and Running a Standard Simulation

- Configuration
  - Edit the config recipe of the justfile, selecting simulation type, control type, and all parameters related to the two.
  - Example config recipes can found in the **JustfileExamples** folder for reference
  - Execute "just help tree" to see the different recipes and their types and sub-types, then produce all of the flags (necessary and optional) by entering "just recipe type subtype". Those flags can then be copied and pasted into the Justfile under the appropriate recipe.
  - Run "just config" to generate the necessary files for the run using the flags from the config recipe
- Run
  - Start the simulation with "just run"

1.2.1 Configuring and Running a Standard Simulation

The first decision to make is whether you would like to run a simulation of a boundary layer (BL) flow or shock-boundary layer interaction flow (SBLI). Note the software doesn't support channel flow right now.

Uncomment the option you would like to run, around lines 36-37.

```
# streams_flow_type := "shock-boundary-layer"
# streams_flow_type := "boundary-layer"
```

Next, examine the config section, the heart of the user interface where simulation and control method parameters are selected. As seen below, there are two offset columns of options. The first contains options pertaining to configuring the simulation itself, such as the mach number and grid specifications, and the second has options pertaining to the control method, which itself is specified at the interface of the two columns, here seen to be learning-based ddpg.

The first column of options are used for all simulations. As previously mentioned, the control strategy (here learning-based) and method (here ddpg) are selected at the interface of the first and second columns. There are three control strategy groups (open-loop, classical, and learning-based) and within each there are a certain number of algorithms that may be chosen. For example, learning-based has the control strategies of ddpg, ppo, and dqn available. The control method and control strategy determine the options the second column is required to contain.

Populate the config recipe of the justfile with flags and values corresponding to the simulation you would like to run

Please use "just help tree" and then "just help recipe type subtype", as well as the justfile examples folder, for a comprehensive set of configuration templates.

```
config:
 echo {{config_output}}

 # 600, 208, 100

 cargo r -- \
  config-generator {{config_output}} {{streams_flow_type}} \
  --steps 6 \
  --reynolds-number 250 \
  --mach-number 2.28 \
  --x-divisions 600 \
  --y-divisions 208 \
            ...
  --use-python \
  --nymax-wr 201 \
  --sensor-threshold 0.1 \
  learning-based ddpg \
      --slot-start 100 \
      --slot-end 149 \
      --train-episodes 2 \
      --training-output {{training}} \
      --eval-episodes 2 \
      --eval-max-steps 6 \
      --eval-output {{eval}} \
      --checkpoint-interval 5 \
      --checkpoint-dir {{checkpoint}} \
      --amplitude 1.0 \
      --learning-rate 0.0003 \
      --gamma 0.99 \
      --tau 0.005 \
      --buffer-size 100000
```

After you have provided the necessary flags to the config recipe in the justfile, return to the terminal and run

```
just config
```

The simulation has been configured and can now be run by executing

```
just run
```

in the terminal.

1.3 Evaluation Run (Relevant for LearningBased Simulations)

- Evaluation
  - Evaluating multiple learned and saved network parameters requires the training phase of a LearningBased algorithm to be skipped.
  - This is done by executing the eval recipe, with the name of the episode of training in which the network parameters were produced as a flag.

LearningBased control strategies have a training phase and evaluation phase. During the training phase, the algorithm parameters being optimized may be saved at different points in the training process. The evaluation phase that is built-in to the standard run (i.e. just run) only use the best performing parameters collected from training, however the user may want to test other parameters for comparison and this can be done by using the evaluation-only runs structured in the eval recipe below.

```
eval:
        cargo r -- run-local \
                --workdir {{results_dir}}/ \
                --config {{config_output}} \
                --database {{database_bl}} \
                --python-mount $STREAMS_DIR/streamspy \
                --eval-only \
                --checkpoint {{checkpoint}}/{{checkpoint_tag}} \
                16
```

The checkpoint parameter is the only parameter that should be modified by the user, specifically checkpoint_tag which can be set to "best", which it will default to if no other tag is provided, "epXXXX", or "initial". Every run has "initial" and "best" parameters to load. Whether epXXXX parameters are saved depends on how you configured your simulation. All available parameters are found in the output directory (STREAMS_DIR/output) the structure of which is described in the next section.

`checkpoint_tag := "best"`
After you change checkpoint_tag flag to "initial", "epXXXX", or "best" run the evaluation-only simulation with the following command executed from the terminal from the streams-utils directory.
`just eval`

Note that an evaluation run will not affect any of the training data but will delete and replace any existing evaluation data.

1.4 Simulation Output

- Output data
  - Generally speaking, distribute_save contains simulation field data
  - LB_GymData and LB_EvalData contain reinforcement learning metrics related to the RL training and evaluation runs, respectively

After a simulation has been run, the results will be stored in the "output" folder found in the following directory
Note: Run echo $STREAMS_DIR to produce the path to streamspy

```
$STREAMS_DIR/streamspy
```

and will contain all of the data generated by the run that has been made accessible to the user.

1.4.1 OpenLoop Output

If an OpenLoop control method is selected the data will be contained in the directory
`$STREAMS_DIR/streamspy/output/distribute_save`

and will contain the five h5 files, mesh.h5, span_averages.h5, trajectories.h5, and time.h5, as well as the standard output produced by STREAmS solver before modification, which is described in the original documentation linked here

`https://github.com/matteobernardini/STREAmS`

The structure of the h5 files are described in the "h5 file schema" section of the appendix linked here A.1.1.

1.4.2 Classical Control Output

Similar to OpenLoop

If a Classical control method is selected the data will be contained in the directory
`$STREAMS_DIR/streamspy/output/distribute_save`

1.4.3 LearningBased Output

Data collected during LearningBased runs are divvied up between the two directories below
`STREAMS_DIR/streamspy/output/LB_GymData`
`STREAMS_DIR/streamspy/output/LB_EvalData`

LB_GymData contains checkpoints folder that houses all of the parameters ("initial", "epXXXX", and "best") saved during training. It also contains training.h5 and evaluation.h5 which have the time, reward, observation, and action from each step of the simulation.

LB_EvalData contains the same h5 files as OpenLoop control (span_averages.h5, mesh.h5, etc.). The data is only collected during evaluation runs because the changing parameters during training make data collection meaningless.

1.4.4 Preserving your data

1. If the run recipe is executed, data from the output folder of the previous run therein may be overwritten.
2. If the eval recipe is run, all data from the previous evaluation will be deleted. The training data will remain.
3. Running analysis, visualization, or modeling recipes (addressed below) will add folders and data files within the output folder but will not alter other data found therein.

1.5 Analyzing a Simulation

- Proper orthogonal decomposition (POD), Dynamic mode decomposition (DMD), and PySensors are the analysis methods that are currently available.
- POD and DMD are modal decompositions whereas PySensors is a software created by the Brunton Lab at Univ. of Washington that determines optimal sensor placement.

All analysis recipes have the basic format seen below. The results_dir flag does not ever need to be modified. The analysis_tag is not used for OpenLoop or Classical runs, but will be ignored there does not need to be removed. The analysis flag is only required when analyzing LearningBased runs. For different analyses, additional flags are required.

```
analysis:
        python3 $STREAMS_DIR/streamspy/Analysis/analysis_main.py \
                --results_dir {{results_dir}} \
                --analysis-tag ep0000 \
```

The analysis recipe uses the the 2D span-averaged datasets saved to either

`$STREAMS_DIR/streamspy/output/distribute_save`

in the case of OpenLoop control, or

`$STREAMS_DIR/streamspy/output/LB_EvalData`

in the case of LearningBased control, and in both cases the results will be housed in a new folder called analysis_results, created in the respective directory. To date (9/23/25), the available analysis methods are proper orthogonal decomposition (POD), dynamic mode decomposition (DMD), and a sensitivity analysis via the PySensors software that may aid with sensor placement (Documentation: https://python-sensors.readthedocs.io/en/latest/api/pysensors.reconstruction.html).

For POD and DMD, you must simply select the evaluation episode (epXXXX) you would like to run the analysis on as well as analysis method (POD or DMD).

```
        --analysis-tag ep0000 \
        --analysis-method POD
```

To run pysensors enter pysensors as the analysis method and also add the flag below, which specifies the number of sensor locations pysensors will return.

```
        --num-sensors 5 \
```

1.6 Visualizing Simulations and Results

The visualization recipe can be used to produce:

- Snapshots and animations of Rho, U, V and W
- Plots of reinforcement learning metrics and important span averaged data

Similar to analysis recipes, all visualization recipes have the basic format seen below. The only flag to consider are the –episode-tag and –vis-type flags. The correct episode number (epXXXX) must be provided when visualizing LearningBased simulations, otherwise the flag is ignored.

```
visualize:
        apptainer exec --nv \
                --bind {{results_dir}}/distribute_save:/distribute_save,{{results_dir}}/input:/input \
                ./streams.sif \
                --episode-tag ep0000 \
                --vis-type animation \
```

The visualizations will be housed in a new folder called visualization_results, created in the output folder, which has the path seen below.

`$STREAMS_DIR/streamspy/output`

To produce a snapshot, add the following flags and the value of –vis-type must be set to snapshot, the variable flag may be rho, u, v, or w, and the snapshot-number should be set to the snapshot you would like to visualize.

```
        --vis-type snapshot \
        --variable w \
        --snapshot-number 99 \
```

To produce an animation, set vis-type to animation and add the variable flag.

```
        --vis-type animation \
        --variable w \
```

To plot reinforcement learning metrics, set vis-type to rl_data

```
        --vis-type rl_data \
```

To plot span averaged data, set vis-type to sa_data

```
        --vis-type sa_data \
```

1.7 Modeling

Modeling workflows will be documented in a future revision.

-

A For POD and DMD, you must simply select the evaluation episode (epXXXX) you would like to run the analysis on as well as analysis method (POD or DMD).

```
        --analysis-tag ep0000 \
        --analysis-method POD
```

To run pysensors enter pysensors as the analysis method and also add the flag below, which specifies the number of sensor locations pysensors will return.

```
        --num-sensors 5 \
```

2 Developer Instructions

The primary value of streamsML is that it can be used to test one's own control and analysis methods. Thanks to its modularity, this can be done by abiding by a flexible format for control and analysis scripts and making minimal additions the supporting Rust code. The required formatting and Rust modifications are described in this section.

2.1 Adding Control Methods

All three control methods require making the same three additions to the Rust code, specifically cli.rs, in order to link the justfile, housing the user-defined parameters you require for your control algorithm, to the python code in which the algorithm will run. This requires simple pattern matching and is possible even if you are unfamiliar with Rust. Beyond that, they differ in how the algorithm itself is added to the Python code.

OpenLoop: Control added directly to jet_actuator.py, Rust code added cli.rs

Classical: TBD

LearningBased: New algorithms encoded in a new script and added to the Control folder. Rust code added to cli.rs.

2.1.1 Justfile and Rust Modification

Let ctrlmethod represent the type of control method you are implementing (i.e. OpenLoop, Classical, or LearningBased), algname represent the name of the new algorithm (e.g. constant, dmdc, ddpg, etc) and param1, param2 represent the user-defined parameters required to run your algorithm (e.g. slot_start, slot_end, amplitude, etc.). Lastly, dtype represents the data type of your parameter (e.g. usize, f64, String).

Step 1: Define the control method, algorithm name, and parameters for your new algorithm.

```
config:
 # rm -rf {{results_dir}}
 # mkdir {{results_dir}}
 echo {{config_output}}

 # 600, 208, 100

 cargo r -- \
  config-generator {{config_output}} {{streams_flow_type}} \
  --steps 6 \
  --reynolds-number 250 \
            ...
  --sensor-threshold 0.1 \
    ControlMethod algname \
      --slot-start 100 \
      --slot-end 149 \
      --param1 user-input \
      --param2 user-input \
      --param3 user-input \
```

Next we modify the Rust code to reflect the newly created algorithm. Open cli.rs ($STREAMS_UTILS_DIR/src/cli.rs) and scroll to the JET ACTUATOR section starting on line 305 and make the three additions corresponding to steps 2-4 boxed on the following page, and finally rebuild the container with the new Rust code (step 5).

```
----------------------------------JET ACTUATOR----------------------------------
```

Step2: Add a new match statement to the jet actuator implementation

```
impl From<JetActuatorCli> for JetActuator {
    fn from(cli: JetActuatorCli) -> Self {
        use JetMethod::*;

        ...

        match cli {
            JetActuatorCli::None =>
                Self { method: algname,
                       strategy: "algname".into(),
                       params: serde_json::to_value(a).unwrap() },
```

Step3: Add your algorithm as a variant to the OpenLoopActuator enum

```
// JET ACTUATOR: OpenLoop Actuator Parameters

#[derive(Subcommand, Debug, Clone, Serialize, Deserialize)]
#[clap(rename_all = "lower")]
pub(crate) enum OpenLoopActuator {
    /// jet actuator description here
    algname(algnameArgs),
}
```

Step4: Add a new struct for your algorithm will all of the required parameters

```
#[derive(Parser, Debug, Clone, Serialize, Deserialize)]
pub(crate) struct algnameArgs {
    #[clap(long)]
    pub(crate) slot_start: usize,

    #[clap(long)]
    pub(crate) slot_end: usize,

    #[clap(long)]
    pub(crate) param1: dtype,

    #[clap(long)]
    pub(crate) param2: dtype,
}
```

Step5: Rebuild your container by re-running the build recipe
`just build`

2.1.2 OpenLoop Control Methods

Most Openloop Control Methods are simple and are therefore added directly to jet_actuator.py. ($STREAMS_DIR/streamspy/jet_actuator.py). Please open jet_actuator.py and complete the following two steps.

Step 1: Add your algorithm name and parameters to the init_actuator function

```
def init_actuator(rank: int, config: Config) -> AbstractActuator:
    jet_config = config.jet

    if jet_config.jet_method_name == "None":
        return NoActuation()

    elif jet_config.jet_method_name == "OpenLoop":

        elif jet_config.jet_strategy_name  == "algname":
            slot_start = jet_config.jet_params["slot_start"]
            slot_end = jet_config.jet_params["slot_end"]
            param1 = jet_config.jet_params["param1"]
            param1 = jet_config.jet_params["param2"]
            return algnameActuator(amplitude, slot_start, slot_end, param1, param2, param3);
```

Step 2: Add the algnameActuator class called from the init_actuator function. It should contain an init method with the standard attributes seen below as well as a step_actuator method where the OpenLoop actuation is defined.

```
class algnameActuator(AbstractActuator):
    def __init__(self, amplitude: float, slot_start: int, slot_end: int, ... , rank: int,
        config: Config, param1:dtype, param2:dtype ):

        utils.hprint("initializing a algname actuator")

        self.slot_start = slot_start
        self.slot_end = slot_end
        self.amplitude = amplitude

        self.actuator = JetActuator(rank, config, slot_start, slot_end)

        self.param1 = param1
        self.param2 = param2

    # returns the amplitude of the jet that was used
    def step_actuator(self, time: float, i:int, agent_amplitude: float = None) -> float:

        adjusted_amplitude = encode your open loop actuation here

        self.actuator.set_amplitude(adjusted_amplitude)
        return adjusted_amplitude
```

2.1.3 Classical Control Methods

Documentation will be added in a future revision.

2.1.4 LearningBased Methods

LearningBased control algorithms are coded in their own module, whose name must equal the learning strategy string used in the Justfile and Rust code.

For example, a deep deterministic policy gradient (DDPG) method is given the string "ddpg" in the just file and Rust code, so the name of the module in which it is coded must be called ddpg.py, and is placed in the $STREAMS_DIR/streamspy/Control directory.

Additionally, if the algorithm is initialized with parameters that are not already found in the kwargs dictionary near the end of main.py, they must be added.

```
    agent_kwargs = {
        # "checkpoint_dir": env.config.jet.jet_params.get("checkpoint_dir"),
        "checkpoint_dir": str(checkpoint_dir),
        "hidden_width": env.config.jet.jet_params.get("hidden_width"),
        "buffer_size": env.config.jet.jet_params.get("buffer_size"),
        "batch_size": env.config.jet.jet_params.get("batch_size"),
        "lr": env.config.jet.jet_params.get("learning_rate"),
        "target_update" :env.config.jet.jet_params.get("target_update"),
        "GAMMA": env.config.jet.jet_params.get("gamma"),
        "TAU": env.config.jet.jet_params.get("tau"),
        "epsilon": env.config.jet.jet_params.get("epsilon"),
        "eps_clip": env.config.jet.jet_params.get("eps_clip"),
        "K_epochs": env.config.jet.jet_params.get("K_epochs"),
    }
```

Lastly, and most critically, the algorithm must implement a class agent that subclasses BaseAgent, providing the choose_action, learn, save_checkpoint, and load_checkpoint methods that are called from main.py. The algorithm can, and almost certainly will, have other classes within it, but these four classes are the interface between the algorithm module and main.py. Refer to ddpg, ppo, or dqn for an example.

```
class agent(BaseAgent):
    def __init__(self, state_dim, action_dim, max_action, ... lr, GAMMA, TAU, checkpoint_dir):

    def choose_action(self, s):

    def learn(self, obs, action, reward, next_obs):

    def save_checkpoint(self, directory: Path, tag: str) -> None:

    def load_checkpoint(self, checkpoint: Path) -> None:
```

Here is a little more information about the rolls each method is required to fulfill inside main.py

- __init__ : Sets up networks, buffers, hyperparameters
- choose_action(observation) → action: Called with a PyTorch tensor of the current observation and expected to return an action compatible with the environment's action space
- learn(obs, action, reward, next_obs) → None: Invoked on each training step with the current observation, the action taken, the received reward, and the next observation; no return value is consumed by the caller
- save_checkpoint(directory, tag) → None: Used to persist model parameters at specified milestones; takes a target directory and a string tag, with no return value used
- load_checkpoint(checkpoint) → None: Loads previously saved parameters from a given path before evaluation; the caller expects no return value

2.1.5 Wrapping additional STREAmS Functions

Users may want to create new functions in order to access different types or locations of simulation data hereto unavailable in Python, and this can be done by adding functions to mod_api.F90

While most of the functions in min_api.F90 give Python access to fundamental STREAmS operations coded in Fortran. mod_api.F90, on the other hand, contains functions to access the the data stored in mod_streams.F90, where all variables used by the Fortran solver are stored.

The functions will have the general form seen below:

```
subroutine function_name(function_outputs, function_inputs) bind(C, name="function_name")
    use iso_c_binding
    use mod_streams, only : selected_variable
    !f2py intent(c) function_name
    !f2py intent(out) function_output
    implicit none

    type, intent(in/out) :: function_in/output

    begin_function_definition
        indexing and manipulation of selected_variable
        function_outputs =
    end_function_definition

end subroutine function_name
```

For example, the function that collects tauw x, which has been used to inform the initial DDPG algorithm aiming to reduce drag reduction, is provided below. Note that it has one function output (tauw_out) and one function input (n), for which the user provided the dimension of the x domain. Also not that an integer i, used for indexing, is created for use in the function.

```
subroutine wrap_get_tauw_x(tauw_out, n) bind(C, name="wrap_get_tauw_x")
    use iso_c_binding
    use mod_streams, only: tauw_x
    !f2py intent(c) wrap_get_tauw_x
    !f2py intent(out) tauw_out
    implicit none
    integer, intent(in), value :: n
    real(8), intent(out) :: tauw_out(n)
    integer :: i

    do i = 1, n
        tauw_out(i) = tauw_x(i)
    end do
end subroutine wrap_get_tauw_x
```

2.2 Adding Analysis Methods

Analysis methods do not use the streamsML container and therefore do not involve Rust. Because of this, new analysis methods can be added by modifying the Justfile and analysis_main.py, and then adding the script encoding your analysis method to the Analysis folder ($STREAMS_DIR/streamspy/Analysis)

Step 1: Add the title of the new analysis method and any additional required parameters to the analysis recipe of the justfile.

The first two parameters are used by analysis_main.py to access 2D span-averaged datasets of a user-defined episode, and therefore likely not have to be changed.

```
analysis:
        python3 $STREAMS_DIR/streamspy/Analysis/analysis_main.py \
                --results_dir {{results_dir}} \
                --analysis-tag ep0000 \
                --analysis-method newmethod \
                --param1 param1 \
                --param2 param2 \
                ...
```

Step 2: In the Analysis folder ($STREAMS_DIR/streams/Analysis), add the module that will contain the new analysis method. It's should be title the same as the newmethod parameter from the justfile.

Step 3: In analysis_main.py, also found in the Analysis folder, import the new module, also, if there are any parameters in the new justfile recipe that are not already in parser, add them.

Note that arguments may be required, otherwise they may have default values, and if the argument is not required and does not have a default value, the argument will be set to None if not provided in the justfile.

```
    from newmethod import run_newmethod

    ...

    def main() -> None:
        parser = argparse.ArgumentParser(description="Run analysis on StreamsML output")
        parser.add_argument("--results_dir", type=Path, required=True, help="Path to output directory")
        parser.add_argument("--analysis-tag", default="", help="Episode tag used for learning based runs")
        parser.add_argument("--analysis-method", default="POD", help="Analysis method to apply")
        parser.add_argument("--param1", type=dtype, required=True, help="Short description")
        parser.add_argument("--param2", type=dtype, required=True, help="Short description")
        parser.add_argument("--param3", type=dtype, required=True, help="Short description")
        ...
        args = parser.parse_args()
```

Step 3: Add a conditional to run run_newmethod(sa_path, output_dir, param1, param2, ...) if analysis-method flag is set to newmethod.

```
        if method == "pod":
            run_pod(sa_path, output_dir)
```

As implied by the instructions, new analysis methods are not required to follow a rigid format. If additional software is required to run the analysis method, it must be installed outside the container and if the addition is permanent, should be added to the requirements.txt file that handles local system imports.

3 Developer Information

Appendix

A.1 File Organization

A.1.1 h5 file schema

Data is generally organized as a large array of the shape

$$(\text{timestep_written},\ \text{vector_component},\ x,\ y,\ z)$$

For `mesh.h5`, `x_grid` contains the stencil in the x direction, `y_grid` is the stencil in the y direction, and `z_grid` is the stencil in the z direction. For example, the config file above specifies that there are 600 x divisions in the grid, and the `x_grid` dataset is $(1, 600)$ in shape.

```
HDF5 "mesh.h5" {
GROUP "/" {
   DATASET "x_grid" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 1, 600 ) / ( 1, 600 ) }
   }
   DATASET "y_grid" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 1, 208 ) / ( 1, 208 ) }
   }
   DATASET "z_grid" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 1, 100 ) / ( 1, 100 ) }
   }
}
}
```

`dissipation_rate`, `energy`, `shear_stress`, and `time` are all scalar quantities saved at every 10 timesteps. Therefore, the time at which the ith index of `energy` was collected is the ith index of `time`.

The `span_average` dataset is in the shape of (NUM_WRITES, vector_component, x, y) and it is averaged over the z direction. The `vector_component` mentioned in the shape is identical to the vector of conserved variables from the paper with density normalization removed. Therefore the components are

$$[\rho, u, v, w, \text{energy}]$$

Indexing the `span_average` dataset in Julia at `[1,2, :, :]` would then provide the span average u velocity at the first timestep.

```
HDF5 "span_averages.h5" {
GROUP "/" {
   DATASET "dissipation_rate" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 5000 ) / ( 5000 ) }
   }
   DATASET "energy" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 5000 ) / ( 5000 ) }
   }
   DATASET "shear_stress" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 5000, 600 ) / ( 5000, 600 ) }
   }
   DATASET "span_average" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 5000, 5, 600, 208 ) / ( 5000, 5, 600, 208 ) }
   }
   DATASET "time" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 5000 ) / ( 5000 ) }
   }
```

```
}
}
```

The trajectories simply contain `dt` information for now since it's helpful for `energy`, `shear_stress`, and `dissipation_rate` to be synced with the span average information.

```
HDF5 "trajectories.h5" {
GROUP "/" {
   DATASET "dt" {
      DATATYPE  H5T_IEEE_F32LE
      DATASPACE  SIMPLE { ( 50000 ) / ( 50000 ) }
   }
}
}
```

B.2 Control Methods

B.2.1 Deep Q Learning (DQN)

B.2.2 Deep Deterministic Policy Gradient (DDPG)

B.2.3 Proximal Policy Optimization (PPO)

C.3 Analysis Methods

C.3.1 Proper Orthogonal Decomposition (POD)

C.3.2 Dynamic Mode Decomposition (DMD)